Confusion regarding Ports at Server Side and Client Side

Q1. Say, I am running flask server in my local and I don't mention the port then will it run on 80 port

No, if you run a Flask server **without specifying a port**, it **will not** run on port 80 by default. Instead, Flask defaults to **port 5000**.

Flask Default Behavior

When you run:
flask run
or
app.run()
Flask starts on:
http://127.0.0.1:5000/

Why Not Port 80?

- **Port 80 requires administrative privileges** (on most operating systems). Running a server on port 80 typically requires sudo (on Linux/macOS).
- Flask is a **development server**, and using port 5000 avoids conflicts with other web services.
-

How to Run Flask on Port 80?

You can explicitly specify it:

flask run --host=0.0.0.0 --port=80
or in Python:

app.run(host="0.0.0.0", port=80)
But if you try this on Linux/macOS, you'll likely get a permission error:

PermissionError: [Errno 13] Permission denied

Solution for Running on Port 80

1. **Run Flask as root (not recommended for security reasons)**:

   sudo flask run --host=0.0.0.0 --port=80

2. **Use a reverse proxy like Nginx or Apache** (recommended for production).

Final Answer

- Flask **defaults to port 5000**, not 80.
- To run on port 80, you **must explicitly set it** and may need **root privileges**.

- In production, use **Nginx/Apache as a reverse proxy** instead of running Flask directly on port 80.

Q2. Say, I am hitting http abc.com , http xyz.com all are going to port 80 at server side

Yes! If you are hitting http://abc.com and http://xyz.com without specifying a port, they will both default to **port 80**on the server side.
However, **how the server handles them** depends on the server configuration. Here's how it works:

1. How Requests Work Internally

1. **DNS Resolution**
   - abc.com resolves to **Server IP X**.
   - xyz.com resolves to **Server IP X (same server)** or **Server IP Y (different server)**.
1. **Client Sends HTTP Request to Port 80**
   - Your browser makes a request:vbnet
     CopyEdit

     GET / HTTP/1.1
   - Host: abc.com
   -

   - The same happens for xyz.com:vbnet
     CopyEdit

     GET / HTTP/1.1
   - Host: xyz.com
   -

1. **Server (Web Server like Nginx/Apache) Listens on Port 80**
   - If abc.com and xyz.com are hosted on **different servers**, each one processes its own request separately.
   - If both domains **resolve to the same server**, the web server (e.g., Nginx, Apache) uses **Virtual Hosts** (or **Server Blocks**) to determine which website to serve based on the Host header.

2. How Does the Server Differentiate Between abc.com and xyz.com?

If abc.com and xyz.com point to the **same server IP**, a web server (e.g., **Nginx**, **Apache**) checks the Host header to decide which content to serve.
Example: Nginx Virtual Hosts Configuration

If abc.com and xyz.com are hosted on the **same** server, you can configure Nginx like this:

```
server {
    listen 80;
    server_name abc.com;
    root /var/www/abc;
}

server {
    listen 80;
    server_name xyz.com;
    root /var/www/xyz;
}
```

- When a request comes for **abc.com**, Nginx serves files from /var/www/abc/.
- When a request comes for **xyz.com**, Nginx serves files from /var/www/xyz/.

3. What If Both Domains Are on Different Servers?

If abc.com and xyz.com resolve to **different IPs**, then:

- abc.com → Routes to **Server A** (IP 12.12.12.1 on port 80)
- xyz.com → Routes to **Server B** (IP 34.34.34.2 on port 80)

Each server independently handles the request.

4. Special Case: If No Virtual Hosts Are Configured

- If both abc.com and xyz.com resolve to the **same IP**, but no virtual host is set up, the web server will **serve the default website** (usually the first one in the configuration).

Q3. So, traffic coming to my client side from server are passing through port 80. Won't it get overwhelmed

Great question! Yes, all **incoming traffic** from the server to your client typically passes through **port 80 (for HTTP)** or **port 443 (for HTTPS)**, but modern networking mechanisms prevent it from getting overwhelmed. Here's how:

1. How Client-Side Ports Work

- While the **server** listens on port **80**, the **client (your browser)** doesn't use port 80 as its receiving port.
- Instead, the client uses a **random ephemeral port** (e.g., 49152–65535) for the response.

Example Connection Process:

1. You request http://abc.com
   - Your **browser sends a request** to abc.com:80 from a random **ephemeral port** (e.g., 52837).
   - Example:
     Client: 192.168.1.10:52837 → Server: 12.12.112.1.13:80

◦

1. The **server sends the response** back to the same client port:

   Server: 12.12.112.1.13:80 → Client: 192.168.1.10:52837
2. The client handles the response and closes the connection.

2. How Port 80 Handles Large Traffic on the Server

Since **port 80 is only the listening port**, web servers handle thousands or even millions of requests using **connection handling mechanisms** like:

1. **Multi-threading & Worker Processes**
   - Web servers (e.g., Nginx, Apache) create **worker processes/threads** to handle multiple requests in parallel.
1. **Load Balancers & Reverse Proxies**
   - Large-scale websites use **load balancers** (e.g., Nginx, HAProxy) to distribute traffic across multiple servers.
1. **Keep-Alive & HTTP/2 Multiplexing**
   - Instead of opening new connections for each request, **Keep-Alive** allows a single connection to be reused.
   - HTTP/2 allows **multiple requests over a single connection**.

3. Won't My Internet Router Get Overwhelmed?

No, because your home/office router uses **Network Address Translation (NAT)** to track and forward responses efficiently.

Example: Multiple Users Behind the Same Router

1. **Client A:**
   - Requests http://abc.com from **port 52837**
   - Router maps: Internal 192.168.1.10:52837 → External WAN_IP:30001 → abc.com:80
1. **Client B:**
   - Requests http://xyz.com from **port 52838**
   - Router maps: Internal 192.168.1.11:52838 → External WAN_IP:30002 → xyz.com:80

Each request-response pair is mapped uniquely, so multiple clients can communicate with the same web server efficiently.